If you have experience building apps with the full .NET Framework or mobile apps with Xamarin, you might have some experience with unit testing frameworks. If you were building apps in Visual Studio, the steps for creating a test project differed among testing frameworks (such as xUnit, NUnit, and MSTest), and running the tests in Visual Studio often required installing a plugin. Similarly, running tests from the command line varied among frameworks.

With the .NET SDK, testing in ASP.NET Core and .NET Core is a first-class citizen, on a par with building, restoring packages, and running your application. Just as you can run `dotnet build` to build a project, or `dotnet run` to execute it, you can use `dotnet test` to execute the tests in a test project, regardless of the testing framework used.

The `dotnet test` command uses the underlying .NET SDK to execute the tests for a given project. This is the same as when you run your tests using the Visual Studio test runner, so whichever approach you prefer, the results are the same.

Test projects are console apps that contain several tests. A *test* is typically a method that evaluates whether a given class in your app behaves as expected. The test project typically has dependencies on at least three components:

- The .NET Test SDK
- A unit testing framework, such as xUnit, NUnit, Fixie, or MSTest
- A test-runner adapter for your chosen testing framework so that you can execute your tests by calling `dotnet test`

These dependencies are normal NuGet packages that you can add to a project, but they allow you to hook in to the `dotnet test` command and the Visual Studio test runner. You'll see an example .csproj file from a test app in the next section.